

A PoC for Portable Decentralized Agents compatible with Web Apps

Abstract

This project focuses on developing a proof of concept that exists out of two parts.

1. A standardized method to define a set of decentralized agents that can exchange messages and data, and also send, accept and reject tasks.
2. A JavaScript package that makes a developer able to instantiate these agents within their web apps, to let them connect to other agents and to expose the functionality of these web apps as the agent's capabilities.

Problem description

While the rise of agents on the web has potential to return the web to a more decentralized structure, it is again a few big organisations that appear to try to dominate this new trend by developing closed source (AI) agents, potentially collecting even more user data, making them more dependent, and so tightening their grip on users even more. User autonomy and control over their own data seems to be reduced.

Proposed solution

We build a PoC for a minimal, portable standard for turning any JavaScript based web app into an agent that can participate in decentralized data exchange, messaging and task sharing. In order to do this, we want to develop an open source function package that:

- helps developers turn their application into an agent that operates in a user defined decentralized agent network
- where data is only shared on terms of the users or agents, without need for a central server

We want to build on solutions that already exist and only contribute at the parts that in our view are missing. The requirements for embedding already existing software in our project are that they:

1. should have proven themselves as being well-tested (while not necessarily production ready)
2. have a prospect of becoming production ready within this year, and also being maintained for the coming two years.

In the description below we offer examples of existing technologies that we could use for functionalities that we think we should not develop ourselves. In order to stay flexible during

development, we need to allow ourselves to divert to alternative pieces of software if we find that more appropriate for our goals. The technologies suggested should therefore be understood as examples or candidate solutions, without any commitment in this stage.\

- Agent definition and information exchange (e.g. A2A protocol)
- Networking: while we strive to implement compatibility with peer-to-peer connectivity, for this PoC we want to use NKN (New Kind of Network), a blockchain solution that organizes peer-to-peer connectivity, as this already implements a lot of the functionality we need
- Storage: for now the storage will be done in local memory (while for the future we see especially high potential in compatibility with Solid Pods)

New Functionality

The main functionality we want to develop, consists of two parts:

1. A portable file in which the user can define:
 - Data source connections + authentication methods
 - Addresses for agent peers / addresses for agent peer discovery platforms
 - Agent network groups: group membership can influence discoverability for other agents, the exposure of peers and connections and they can be used to manage permissions
2. A portable package that a developer can use to turn an app into one or more agents that are capable of:
 - **Connecting**
 - connecting to data sources
 - connecting to agent peers
 - **Discovery**
 - discovering peers
 - exposing their capabilities to peers (e.g. connections and app functions)
 - **Communicating**
 - messaging
 - exchanging data (requests) with peers
 - submit + accept/reject tasks to the network
3. In order to integrate the agent with the app, a developer must be able to redefine its app functions as agent capabilities. For this we want to develop a (for now JavaScript) package that can be imported by developers which provides ways to:
 - load an agent defined by the user
 - initialize an agent based on the properties file
 - define app-functions as agent capabilities
 - map agent task parameters to function arguments

This way, users can flexibly connect to their agent network from different places, while developers can create apps that can add new functionality to this network.

Technical approach

All parts will be made public under the MIT-license.

1. Design for Agents Properties File

This could be an encrypted yaml or json file that contains:

- agent ID's
- pointer to key/passwords/token vault
- list of connections to:
 - db's
 - api's
 - networks
 - other agent IDs
 - etc
- agent group ID's
- optional: define roles to organize these connections/credentials/groups
- optional: list of approved app-IDs (or link to list)
- permissions
 - for sharing capabilities to group members
 - for sharing peers to group members

2. Optional: Simple web app for creating agent properties file

- Easy to make, useful for testing

3. Agent loading package

1. Runtime

1. load agent properties file
2. initialize agent based on the file (e.g. A2A protocol)
3. send messages to other agents
4. exchange data with other agents
5. send tasks to the network

2. Integration functions for application

1. connect app functions to agent capabilities
2. connect task parameters to function arguments

3. Networking

1. connect to other devices (e.g. New Kind of Network (NKN), in the future preferably p2p technology such as WebRTC)

2. discover given agents on connected devices
3. discover new agents by peer discovery
4. Coordination
 1. local cache of known agents and capabilities
 2. task-capability matcher
 3. procedure for accepting/rejecting a task from the network

Scope of the PoC

What we don't expect in this phase:

- exhausting design of agent properties file. For example no implementation yet of connection to external data sources such as API, databases or Solid Pods
- gradual/layered agent discovery: for now agents just share the whole peer or not at all (dependent on the permission)
- efficient and fluent task sharing
- complete safety
- complete documentation
- automatic updater of the internal model of the network (Local State Synchronizer)
- Production level safety
 - As we want to build onto already existing and trusted solutions, there are already a lot of safety measurements included
 - However, as this is still a proof of concept, there will be safety issues with our first version. The biggest potential leak in our opinion is that currently there is no widely adopted way to prevent web apps from sending user data to any third party - as this is common in the modern web. For this PoC we will not address this issue, as we think this should be implemented at the browser level, which is outside of the scope of this project

What we do expect:

- a minimal working prototype package that
 - can be used to define agents having:
 - accordance with the a2a protocol
 - connects local js functions to agent capabilities
 - at least one functioning connection method to another agent
 - some way to share a peer to another agent
 - a basic task sharing function (submitting/accepting/rejecting)
 - a basic messaging function
- results of small scale testing which only covers whether or not the functionality described above works in principle
- basic documentation that makes a developer capable of testing it

In conclusion, at the end of the project, two browser-based agents can:

- discover each other
- exchange capabilities

- message other agents
- share data with other agents
- submit a task to the agent network
- reject/accept/execute a simple task given by another agent

Roadmap

Phase 1

- browser based agent that is able to connect to another browser based agent
 - defined by standardized protocol (e.g. A2A protocol)
 - exploration of connection method (e.g. NKN)
 - only local storage
 - hard-coded connection method (by ID)
 - simple messaging function
- first version of design of agent properties file

Phase 2

- first version of peer discovery protocol
- first version of capability exchange (no permission checks or whatsoever)

Phase 3

- first version of local state map (without regular updating)
- first version of functions for task submission/rejection/acceptance

Phase 4

- design of agent properties file
- evaluation and improvement of main functions
 - maybe add layered discovery function
- documentation
- demo

Future phases should focus on:

- expanding agent functionality:
 - connections to external sources
 - connection through p2p
- improving the task acceptance negotiating protocol
- implementing layered agent discovery (so no direct exposure, only through verification or whatever)
- finding a case for testing in real life
- browser-level control on user data

Team

Frits de Roos:

- BA philosophy, BSc Artificial Intelligence
- Former functions:
 - data analyst at Fries Planbureau (Frisian Planning Bureau)
 - data scientist / machine learning engineer at start-up (Wastewatchers) focused on developing prediction models for the revenue of food services.
- Strong experience with Python
- Strong experience with task-based distributed computing (but centralized, using Prefect)
- In short:
 - books + discussions raised my interest in reinforcing local communities / reinventing the idea of the commons
 - working at the start-up helped me gain experience in distributed computing and independent project management / system design
- Hobby, partially triggered by both jobs: thinking about digital and societal systems to reduce dependency on big companies. This idea started as some kind of neighborhood question/help app
- Willing to spend 24-32 hours/week, while trying to start a business (8-16 hrs/week)

Address:

Irislaan 62a

9713RL Groningen

Marco Gunnink:

- MSc Computing Science
- Currently working as a software engineer at an IoT company (Twilight)
- Former functions:
 - Teaching assistant for several courses in CS
 - Software developer for University of Groningen
- Strong experience in JavaScript, computer networking
- Long-standing interest in free/libre/open-source software
- Willing to spend 8-16 hours per week on this project, while still working in his current job (32 hours/week)

Address:

Populierenlaan 1 - 7

9741HA Groningen

Budget

A suggestion for the budget allocation:

- milestone 1: €3500,- (100 hrs, €35/h)
- milestone 2: €9100,- (260 hrs, €35/h)

These budgets are negotiable

Milestone 1: Basics for running agent

1. Runtime
 - a. Initialize agent
 - b. Send messages to other agents
2. Networking
 - a. Connect to other devices
 - b. Discover given agents on connected devices
 - c. Discover new agents by peer discovery

Milestone 2: Functioning demo of PoC

1. Design of Agent Properties File
2. Runtime
 - a. exchange data with other agents
 - b. send tasks to the network
3. Integration functions for application
 - a. connect app functions to agent capabilities
 - b. connect task parameters to function arguments
4. Coordination
 - a. local cache of known agents and capabilities
 - b. task-capability matcher
 - c. procedure for accepting/rejecting a task from the network
5. Documentation for developer testing

Risks & uncertainties

- Agent Protocols such as A2A are not yet fully developed for production
- Developing browser based p2p networking can face complexities due to browser limitations and NAT

This is why we focus on a proof-of-concept for now, so we gain well-informed insights on how to continue this project.